Beyond Genre Labels: Exploring Genre Overlap and Song Popularity Through Knowledge Graphs

Final report Knowledge and Data 2026

Ishita Dubey - ean853

Date: [16-10-2026]

1 Introduction

1.1 Goal, motivation and users

Music genres are commonly used to categorize songs, but the boundaries between genres are not always clear. A single song may belong to multiple genres because of similarities in musical style and characteristics. Understanding these overlaps can provide insight into how music is classified and whether genre diversity is associated with song popularity.

The goal of this project is to investigate genre overlaps and their relationship with song popularity using a knowledge graph. The analysis focuses on five genres: pop, rock, alt-rock, indie, and indie-pop. Using a Spotify tracks dataset, the project explores which genres share the most songs and whether tracks associated with multiple genres have different popularity scores compared to those associated with only one of the selected genres.

The analysis is intended for music researchers, streaming-platform analysts, and listeners interested in music classification. By representing songs, artists, and genres as interconnected entities, the knowledge graph provides a structured way to explore relationships between musical categories. SPARQL queries and visualizations will be used to investigate these relationships, while ontology-based reasoning will explore additional connections that can be inferred from the represented knowledge.

1.2 Competency Questions

The project addresses three competency questions:

CQ1: Which pairs of the five selected genres share the highest proportion of tracks?

CQ2: How does average popularity differ between single-genre and multi-genre tracks?

CQ3: Which tracks can be inferred as multi-genre or popular using OWL reasoning?

2 Design of Ontology

2.1 Methodology

The ontology was developed using a combination of a top-down and bottom-up approach. The top-down approach involved identifying the main concepts needed to answer the competency

questions, while the bottom-up approach involved examining the Spotify dataset to understand which information and relationships were available.

The development began by identifying concepts such as MusicTrack, Artist, Album and Genre. These were organized into a class hierarchy in Protégé, with additional subclasses introduced to represent different track classifications. Object properties were used to connect tracks to artists, albums and genres, while data properties represented attributes such as track names and popularity scores. OWL restrictions were introduced to support logical reasoning. The ontology was also partially aligned with Schema.org to connect its music-related concepts to an existing vocabulary. Finally, the ontology was tested using the HermiT reasoner to check whether track classifications could be inferred from the available data.

2.2 Conceptualisation

The ontology was designed to represent relationships between music tracks, artists, albums and genres, focusing on genre overlap and song popularity. While the Spotify dataset stores information in a tabular format, a knowledge graph makes these connections explicit and allows them to be explored through SPARQL queries and logical reasoning.

The central class is MusicTrack, representing individual recordings identified by their Spotify track IDs. Each track connects to an Artist through performedBy, an Album through belongsToAlbum, and one or more Genre individuals through hasGenre. The data properties hasTrackName and hasPopularity store track attributes. This structure allows tracks to belong to multiple genres without creating duplicate individuals.

The ontology also introduces subclasses to represent different track classifications. MultiGenreTrack describes tracks associated with at least two distinct genres, while SingleGenreTrack represents tracks associated with only one selected genre. PopularTrack and LowPopularityTrack distinguish tracks using a popularity threshold of 70.

OWL restrictions enable automatic classification. For example, a track associated with two distinct genres can be inferred as a MultiGenreTrack, while a popularity score of at least 70 allows classification as a PopularTrack. Additional concepts, including GenreOverlappingArtist and MusicRelease, extend the domain beyond individual tracks.

Overall, this conceptual model provides the foundation for analyzing genre overlap, comparing popularity and demonstrating ontology-based reasoning.

2.2.1 Classes

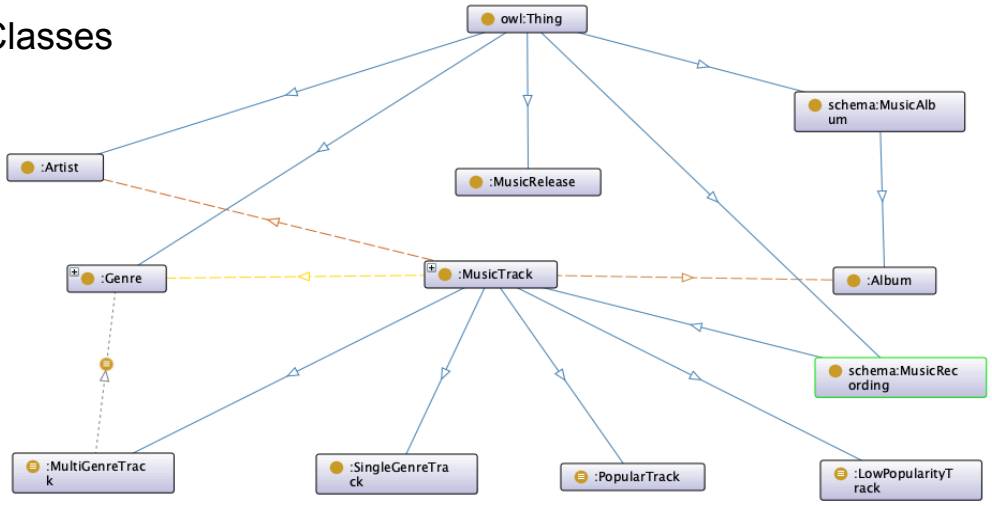


Figure 1. Conceptual structure of the Spotify genre ontology, including class relationships and alignments with Schema.org

Class	Description
MusicTrack	Represents an individual Spotify track, identified using its unique track ID.
Artist	Represents a musician or musical group associated with a track.
Album	Represents an album to which a track belongs.
Genre	Represents a music genre used to classify tracks.
MultiGenreTrack	A subclass of MusicTrack representing tracks associated with at least two distinct genres.
SingleGenreTrack	A subclass of MusicTrack intended to represent tracks associated with only one of the five selected genres.
PopularTrack	A subclass of MusicTrack representing tracks with a popularity score of 70 or higher.
LowPopularityTrack	A subclass of MusicTrack representing tracks with a popularity score below 70.
GenreOverlappingArtist	Represents artists whose tracks are associated with multiple genre categories.
MusicRelease	Represents the broader concept of a published music release.

2.2.2 Properties

The ontology contains five properties: three object properties and two data properties. Object properties represent relationships between individuals, while data properties describe attributes using numerical or textual values.

Property	Type	Domain	Range	Description
performedBy	Object	MusicTrack	Artist	Connects a track to its performing artist.
belongsToAlbum	Object	MusicTrack	Album	Identifies the album associated with a track.
hasGenre	Object	MusicTrack	Genre	Connects a track to one or more music genres.
hasPopularity	Data	MusicTrack	xsd	Stores the track's popularity score.
hasTrackName	Data	MusicTrack	xsd	Stores the name of the track.

2.2.3 Restrictions

The ontology includes three OWL restrictions to classify music tracks based on their genre memberships and popularity scores. These restrictions were defined using equivalent class expressions in Protégé.

MultiGenreTrack - MusicTrack and (hasGenre min 2 Genre)

This restriction defines a MultiGenreTrack as a music track associated with at least two distinct genres. It is central to the project because it allows tracks belonging to multiple categories to be identified automatically. For example, Mr. Brightside by The Killers is associated with both Rock and AltRock, allowing it to be inferred as a MultiGenreTrack. The five genre individuals were declared distinct to ensure that the cardinality restriction works correctly.

PopularTrack - MusicTrack and (hasPopularity some xsd:integer[>= 70])

This restriction defines a PopularTrack as a music track with a popularity score of at least 70. The threshold was selected as a project-specific classification criterion. For example, Mr. Brightside has a popularity score of 86 and can therefore be inferred as a PopularTrack.

LowPopularityTrack - MusicTrack and (hasPopularity some xsd:integer[< 70])

This restriction identifies tracks with popularity scores below 70. It provides a complementary classification to PopularTrack and allows tracks to be categorized according to their recorded popularity values.

2.3 Alignment with Existing Ontologies

To improve the interoperability of the Spotify ontology, a partial alignment was established with Schema.org, an existing vocabulary used to describe structured information across different domains. Schema.org was selected because it includes concepts specifically related to music, making it suitable for the classes represented in this project.

Two alignments were implemented using the rdfs:subClassOf relationship.

Local Class	External Class	Alignment
MusicTrack	schema	rdfs
Album	schema	rdfs

2.4 Formalization / Implementation

The Spotify ontology was implemented using Protégé, with RDF, RDFS and OWL used to formally represent the concepts and relationships defined during conceptualisation. The completed ontology was exported in Turtle (.ttl) format, providing a structured and machine-readable representation of the music domain. The main concepts, including MusicTrack, Artist, Album and Genre, were defined as OWL classes. Subclass relationships were established using rdfs:subClassOf, allowing more specific classifications such as MultiGenreTrack and PopularTrack to inherit from MusicTrack.

Relationships between individuals were implemented using owl:ObjectProperty, while attributes containing numerical or textual values were defined using owl:DatatypeProperty. Domain and range declarations were used to specify how properties relate to the different

classes. For example, hasGenre connects MusicTrack individuals to Genre individuals, while hasPopularity stores integer popularity values. OWL equivalent class expressions were introduced to support automatic classification based on genre membership and popularity thresholds. The five selected genres were represented as distinct individuals of the Genre class, allowing the cardinality restriction for MultiGenreTrack to distinguish between different genre memberships.

The ontology also includes subclass alignments with Schema.org's MusicRecording and MusicAlbum classes. These were implemented using external IRIs to connect the local ontology to an existing vocabulary.

The resulting ontology was saved as Ontology.ttl, separately from instances.ttl, which contains the imported Spotify data. Together, these two files form the complete knowledge graph used for reasoning and analysis.

3 Adding instances

The instances for the knowledge graph were created using the Spotify Tracks Dataset published by maharshipandya on Hugging Face. The original dataset contains approximately 114,000 records across 114 music genres. For this project, the dataset was filtered to five genres: pop, rock, alt-rock, indie and indie-pop, resulting in 5,000 records.

The data preparation was carried out in Python using Pandas in a Jupyter notebook. The filtered dataset contained 4,088 unique Spotify track IDs, meaning that some tracks appeared multiple times under different genre labels. Instead of treating these repeated entries as separate songs, each unique track ID was used to create a single MusicTrack individual. All associated genres were then connected to that individual using the hasGenre property. This preserved the overlapping genre relationships needed for the analysis.

During data cleaning, 36 tracks were found to have conflicting popularity scores across their records. To ensure that each track had one representative popularity value, the average score was calculated and rounded to the nearest integer. The filtered dataset was also checked for missing values in the track ID, track name, artist and album fields, with no missing values found in these columns.

The cleaned data was converted into RDF using the RDFLib Python library. Unique URIs were generated for tracks using their Spotify IDs, while consistent identifiers were created for artists and albums. The properties performedBy, belongsToAlbum, hasGenre, hasTrackName and hasPopularity were used to represent the information as RDF triples.

The final instance graph contains 4,088 music tracks, 1,619 artist individuals and 3,419 album individuals. It also contains 4,989 distinct track–genre relationships. The resulting RDF data was exported to instances.ttl, which can be loaded together with Ontology.ttl to form the complete knowledge graph.

This conversion process allows the original tabular data to be represented as interconnected entities, making it possible to analyze genre overlaps and popularity using SPARQL queries and ontology-based reasoning.

4 The resulting knowledge graph

The resulting knowledge graph combines the ontology developed in Protégé with the Spotify instances generated using Python and RDFLib. The ontology defines the classes, properties, and logical restrictions, while the instance data contains the actual music tracks, artists, albums, and genre relationships. Together, these components provide a structured representation of the selected Spotify music data.

The instance graph contains 31,762 RDF triples, representing 4,088 unique music tracks, 1,619 artist individuals, and 3,419 album individuals. The tracks are associated with five selected genres through 4,989 distinct genre relationships. These relationships are particularly important because some tracks appear under multiple genre categories

Component	Count
Music tracks	4,088
Artists	1,619
Albums	3,419
Selected genres	5
Track–genre relationships	4989
RDF instance triples	31,762

Each track is represented by a unique URI based on its Spotify track ID. This prevents repeated entries in the original dataset from creating duplicate track individuals. Instead, tracks appearing in multiple genres are connected to several Genre individuals through the hasGenre property.

For example, Mr. Brightside by The Killers is represented as a single MusicTrack individual connected to both Rock and AltRock. It also contains a popularity score of 86 and relationships to its associated artist and album. This illustrates how the knowledge graph can combine different types of information about one track.

The completed knowledge graph supports the project's competency questions by allowing genre overlaps and popularity differences to be examined through SPARQL queries. It also supports logical reasoning, which can identify track classifications that were not explicitly assigned in the instance data.

5 Meaningful Inferences

Inference 1: MultiGenreTrack - The MultiGenreTrack class was defined using the following equivalent class expression:

MusicTrack and (hasGenre min 2 Genre)

This allows HermiT to identify tracks associated with at least two distinct genres. For example, Mr. Brightside by The Killers is connected to both Rock and AltRock in the knowledge graph. Since these genres were declared as different individuals, HermiT inferred that Mr. Brightside belongs to the MultiGenreTrack class.

This classification was not explicitly assigned to the track in the instance data. Instead, it was derived from the existing hasGenre relationships.

Inference 2: PopularTrack - The PopularTrack class was defined using the following restriction:

MusicTrack and (hasPopularity some xsd:integer[>= 70])

This allows tracks with popularity scores of 70 or higher to be classified as PopularTrack. Mr. Brightside has a recorded popularity score of 86, which satisfies this condition. HermiT therefore inferred that the track also belongs to the PopularTrack class. These results demonstrate that one track can belong to multiple inferred classes depending on its properties. The two classifications are independent: a track does not need to be popular to belong to multiple genres, and a popular track does not necessarily belong to multiple genres.

6 Relevant SPARQL queries

SPARQL queries were used to retrieve and analyze information from the Spotify knowledge graph. The queries were executed in Jupyter Notebook using RDFLib, allowing relationships between music tracks, genres, and popularity scores to be examined directly from the RDF data.

Query 1 (Genre Overlap (CQ1)) - The first query identifies pairs of genres that share the same music tracks. It retrieves tracks associated with two different genre individuals and counts the number of distinct tracks shared by each pair.

```
SELECT ?genre1 ?genre2
    (COUNT(DISTINCT ?track) AS ?sharedTracks)
WHERE {
  ?track a spotify:MusicTrack .
  ?track spotify:hasGenre ?genre1 .
  ?track spotify:hasGenre ?genre2 .
  FILTER(STR(?genre1) < STR(?genre2))
}
GROUP BY ?genre1 ?genre2
ORDER BY DESC(?sharedTracks)
```

The FILTER condition prevents the same genre pair from appearing twice in reverse order, while COUNT(DISTINCT ?track) ensures that each shared track is counted only once. The results showed that Indie and IndiePop had the highest overlap, sharing 579 tracks. AltRock and Rock followed with 164 shared tracks. Jaccard similarity was subsequently calculated to account for the total number of distinct tracks associated with each pair. Indie and IndiePop had an overlap of 40.80%, compared with 8.94% for AltRock and Rock.

(Query 2: Popularity Comparison (CQ2)) -The second query compares average popularity scores between tracks associated with one genre and tracks associated with multiple genres.

```
SELECT ?genreGroup
    (COUNT(?track) AS ?trackCount)
    (AVG(?popularity) AS ?averagePopularity)
WHERE {
  {
    SELECT ?track ?popularity
        (COUNT(DISTINCT ?genre) AS ?genreCount)
    WHERE {
      ?track a spotify:MusicTrack .
      ?track spotify:hasGenre ?genre .
      ?track spotify:hasPopularity ?popularity .
    }
    GROUP BY ?track ?popularity
  }

  BIND(
    IF(?genreCount >= 2, "Multi-genre", "Single-genre")
    AS ?genreGroup
  )
}
GROUP BY ?genreGroup
```

The nested query counts the number of distinct genres associated with each track. The BIND expression then divides the tracks into single-genre and multi-genre groups. Finally, AVG calculates the average popularity score for each group. The results identified 807 multi-genre tracks with an average popularity score of 37.83, compared with 3,281 single-genre tracks with an average score of 34.98. This suggests that tracks associated with multiple selected genres have slightly higher popularity scores within the dataset. However, the difference is descriptive and does not establish a causal relationship.

Connection to CQ3 - The third competency question focuses on inferred track classifications. Unlike the first two questions, which were investigated using SPARQL aggregation, the reasoning component was evaluated using HermiT in Protégé. OWL restrictions were used to infer whether tracks qualified as MultiGenreTrack or PopularTrack. The inferred classifications were inspected using real Spotify instances, as described in Section 5.

Together, these queries and reasoning results demonstrate how the knowledge graph supports both data retrieval and semantic inference.

7 Data Science pipeline

The data analysis pipeline was developed in Jupyter Notebook using Python, RDFLib, Pandas and Matplotlib. The ontology and Spotify instances were loaded into an RDFLib graph, where SPARQL queries retrieved information about genre relationships and track

popularity. The results were converted into Pandas DataFrames for further calculations and visualization.

Genre Overlap Analysis

The first analysis examined overlaps between the five selected genres. SPARQL identified shared tracks for each genre pair, while Jaccard similarity measured the proportion of shared tracks relative to their combined unique tracks. Indie and IndiePop showed the highest overlap at 40.80%, sharing 579 tracks. AltRock and Rock followed at 8.94%, with 164 shared tracks, while Indie and Rock had an overlap of 4.83%. The smallest overlap was between AltRock and Pop, which shared only one track. These findings suggest that some genre boundaries overlap considerably more than others.

Popularity Comparison

The second analysis compared popularity between single-genre and multi-genre tracks. SPARQL grouped tracks by their number of distinct genre memberships and calculated their average popularity scores. The 807 multi-genre tracks had an average popularity of 37.83, compared with 34.98 for the 3,281 single-genre tracks. This difference of 2.85 points suggests a small positive association between genre overlap and popularity, although it does not establish causation.

Visualisation

Two Matplotlib bar charts were created. The first displays Jaccard similarity percentages across all ten genre pairs, highlighting the substantial overlap between Indie and IndiePop. The second compares average popularity scores between single-genre and multi-genre tracks. Together, these visualizations demonstrate how RDF data can be retrieved using SPARQL, processed in Python and presented for meaningful comparison.

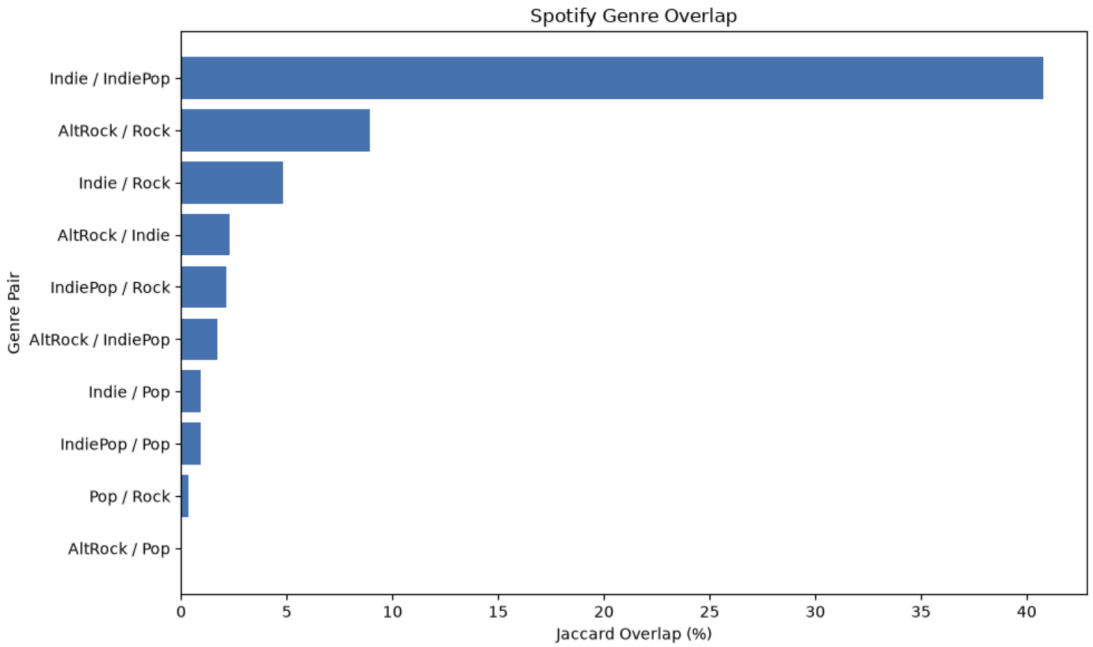


Figure 2. Jaccard overlap percentages between the five selected Spotify genres.

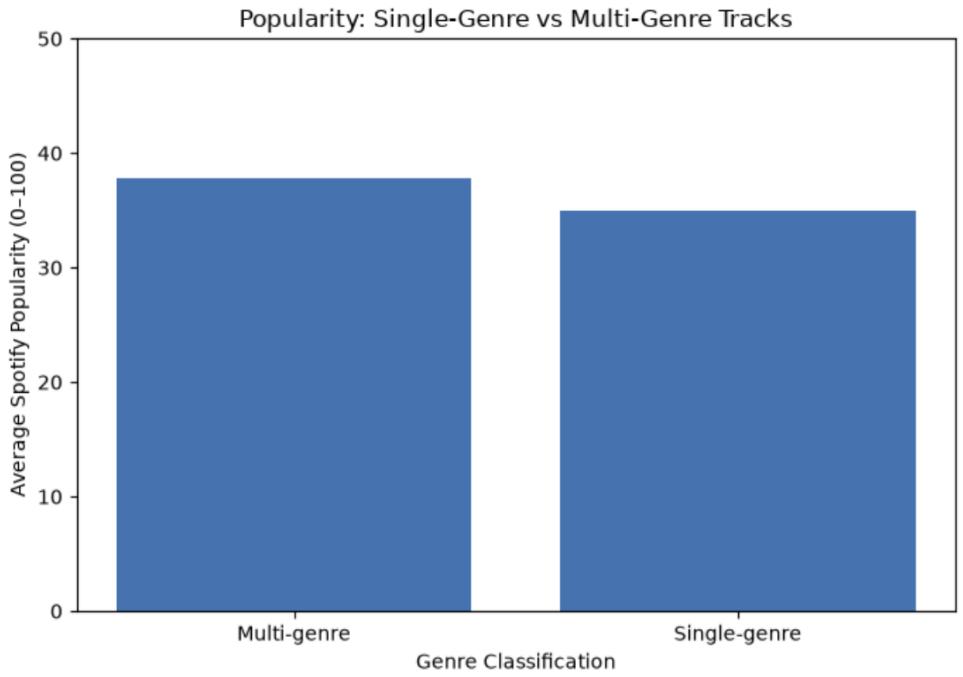


Figure 3. Average popularity scores of single-genre and multi-genre Spotify tracks

8 Reflection and Justification

This project helped me understand how knowledge graphs can represent relationships between music tracks and genres. The main challenges were designing the ontology in Protégé, handling duplicate Spotify records, and ensuring consistent identifiers between Python and Protégé. Running HermiT on the complete dataset was also difficult, so a smaller sample was used for reasoning. The analysis was limited to five genres, and the popularity results do not establish causation. Future improvements could include more genres and additional relationships. Generative AI was used to assist with troubleshooting Python and Protégé errors encountered during the project, as well as proofreading the written report at the end.

9 References

maharshipandya. (n.d.). Spotify Tracks Dataset. Hugging Face. https://huggingface.co/datasets/maharshipandya/spotify-tracks-dataset
Schema.org. (n.d.). MusicRecording. https://schema.org/MusicRecording
Schema.org. (n.d.). MusicAlbum. https://schema.org/MusicAlbum
Stanford University. (n.d.). Protégé ontology editor. https://protege.stanford.edu/
RDFLib contributors. (n.d.). RDFLib: Python library for working with RDF. https://rdflib.readthedocs.io/